Early Warning System for Financial Fraud Detection

Amal Almutairi , Nada Alhusaini , Noorah Alsharif , Nouf Alkhudairi , Rand Alamri

Supervised by: Dr. Safa Habibullah

Department of Information Systems Technology, College of Computer Sciences Engineering, University of Jeddah, Saudi Arabia

1. Abstract

With the rapid growth of online transactions, financial fraud has become a major concern due to evolving attack techniques. Traditional rule-based systems often fail to detect sophisticated fraud, leading to false positives and decreased customer trust. This project proposes an early warning system that analyzes historical transaction patterns to understand behavioral changes over time. The system leverages machine learning to detect anomalies, while incorporating explainable AI techniques to justify predictions and improve collaboration between financial and technical teams. This enhances both detection accuracy and transparency

2. Introduction

With the spread and growth of online transactions, the risk of fraud and the exploitation of security vulnerabilities has increased. This may damage the reputation of banking and financial systems and reduce customer confidence.

Traditional fraud detection systems rely on predefined rules to identify suspicious transactions. Although these methods have been useful, they may fail to detect sophisticated fraud that surpasses simple rule-based detection. This can lead to false positives, causing disruptions and inconvenience for customers.

To address these challenges, this project introduces an early warning system that monitors transaction patterns over time by detecting irregular behaviors. This approach enhances fraud detection accuracy while minimizing false alarms. The system must also be flexible, capable of learning from new data, and adaptable to continuous changes in emerging fraud techniques.To improve the efficiency of collaboration between finance and technology professionals, the model will include an explainable framework explaining why certain transactions are marked as fraudulent. This will enhance transparency and enhance trust in the system, ensuring that professionals in the financial sector can validate and understand the logic behind fraud classifications.

3. Background

Fraud detection in financial transactions has been an ongoing challenge due to the dynamic and evolving nature of fraudulent activities. Traditional systems were built upon static rule-based mechanisms that define suspicious behavior using fixed thresholds or manually created conditions. While these systems have served as a foundational step in fraud prevention, they often lack the flexibility needed to adapt to new and sophisticated fraud patterns [1].

As technology advanced, so did the complexity of fraud, making it more difficult for static rules to effectively detect all threats. This led to the emergence of machine learning and data-driven approaches that analyze historical transaction behavior to uncover hidden patterns and anomalies. Recent advancements in deep learning techniques, particularly Long Short-Term Memory (LSTM) networks, have enabled models to learn sequential patterns, making them ideal for detecting subtle and delayed fraudulent behaviors. Moreover, integrating explainability tools such as SHAP values into these models allows analysts to understand the reasoning behind fraud predictions, enhancing trust and interpretability in financial environments.

This background lays the foundation for the proposed system, which combines explainable machine learning with behavioral analysis to build a more robust and adaptive fraud detection framework [1].

4. Related Work

Financial fraud detection has increasingly relied on AI and machine learning due to the limitations of traditional rule-based systems. [2] evaluates various models, including neural networks (such as CNNs and LSTMs), decision trees, and ensemble methods, and finds that neural networks achieve the highest fraud detection accuracy. However, the study relies on historical transaction data, limiting real-time fraud prevention. [3] propose a deep ensemble learning model using Multivariate Time-Series Data, integrating RNN, LSTM, CNN, Transformers, and GNNs. Their findings show that LSTM models excel in detecting sequential fraud patterns, while GNNs enhance anomaly detection. However, their approach remains post-transactional, lacking real-time fraud prevention.

To address the interpretability challenge in fraud detection models, [4] proposed *TimeTrail*, a framework that leverages temporal correlation analysis to detect fraudulent transaction patterns in a more interpretable manner than traditional black-box deep learning models. This approach benefits financial institutions that require clear justifications for classifying transactions as fraudulent.

In a similar effort, [5] introduced a deep learning approach for fraud detection in financial data with limited labels. The study utilizes SHAP (SHapley Additive Explanations) to provide interpretability, enabling financial analysts to understand the influence of each variable on model predictions. The results demonstrated that integrating explainability techniques enhances fraud detection accuracy while providing greater transparency in decision-making processes.

The Interleaved Sequence RNNs for Fraud Detection study explores the use of recurrent neural networks (RNNs) to improve fraud detection in financial transactions, utilizing large-scale datasets from major institutions. The Anomaly and Fraud Detection Using ARIMA study focuses on time-series anomaly detection with the ARIMA model, using a smaller dataset provided by NetGuardians. The RNN approach relies on a GRU-based deep learning model to process interleaved sequences of transactions dynamically, achieving high recall and operating efficiently in real-time. In contrast, the ARIMA model applies statistical time-series forecasting to detect anomalies based on deviations from expected spending behavior and outperforms other methods like K-means and Isolation Forest in identifying fraudulent activities. Both studies aim to enhance fraud detection accuracy while reducing false positives. The RNN approach continuously learns and adapts to evolving fraud patterns, making it more effective in dynamic environments, while the ARIMA model focuses on historical trends and provides greater explainability.[6][7]

Hilal, Gadsden, and Yawney (2022)[8, 9, 10] focused on the study of different machine learning and deep learning models for anomaly detection in financial fraud in the paper Financial Fraud: A Review of Anomaly Detection Techniques and Recent Developments. This paper states that the Random Forest (RF) algorithm performed particularly well and robustly in fraud detection especially in the recall and precision stages. Other deep learning models such as CNN, LSTM, and Autoencoders have been shown to be more efficient in detecting novel fraud patterns but are computationally expensive. The study also confirmed that these models can be much better at fraud detection if the features and data balancing techniques are chosen appropriately.

the use of big data analytics in financial risk warning systems using Hadoop, Spark, Storm, and deep learning is studied. The article states that no model is better than the other, but each model helps improve risk assessment and prediction accuracy. The main difference lies in the data processing methods used. Storm is a real-time stream, while Hadoop and Spark are distributed computing. The effectiveness of risk warning increases in parallel with diagnosis using differential evolution. [11, 12, 13, 14]

Financial fraud detection using time-series analysis has been extensively studied with machine learning and deep learning techniques. [15] proposed an LSTM with Attention Mechanism for credit card fraud detection, achieving superior accuracy and recall over traditional models, while [16] introduced a PCA-NN model that leverages principal component analysis and neural networks for anomaly detection in financial time series. Despite their effectiveness, these studies primarily focus on historical data classification and batch processing, lacking real-time fraud prevention capabilities. From the studies we reviewed, it is evident that most fraud-detection systems focus on analyzing historical transaction data and identifying fraud after it has already occurred. Although these methods are effective for analytical purposes, they do not facilitate real-time

fraud prevention. Very few studies provide instant alerts or a system capable of warning before a transaction is completed. Consequently, there remains a gap in the research: a simple and practical early-warning system that monitors transactions in real time and immediately flags suspicious activity is still lacking. In contrast, our research aims to develop an early-warning system that integrates real-time monitoring, predictive analytics, and instant alerts, enabling proactive fraud detection and prevention prior to the completion of transactions. Our project seeks to address this gap by building a system that continuously monitors transactions, predicts potential fraud early, and issues alerts before the transaction is finalized.

Table 1: Comparison of Selected Studies

Ref.	Dataset	Model	Best Results	Similarities	Differences
[2]	Financial transaction data	Decision Trees, RF, NN, NLP, Graph Analysis	High accuracy, anomaly detection, real-time monitoring	Focuses on multiple fraud types	Uses NLP and graph analysis; our study uses behavioral analysis
[3]	Multivariate time-series data	RNN, LSTM, CNN, Transformer, GNN	Improved anomaly detection accuracy	Uses historical transaction patterns	No early warning system, detects fraud post-transaction
[5]	Limited labeled financial transaction data	Deep learning with explainability (SHAP)	Improved fraud detection with high interpretability	Focuses on anomaly detection	Introduces SHAP explainability, unlike black-box models
[4]	Financial transaction data (multi-source)	Temporal Correlation Analysis + Time-Series ML	Enhanced fraud pattern detection with high interpretability	Focus on fraud detection	Emphasizes transparency over deep learning
[6]	Credit card data (NetGuardians SA, 11,471 transactions)	ARIMA (time series)	Better than K-means and Isolation Forest	Anomaly detection	Small data, no early detection focus
[7]	European financial data (1B and 4B txns)	GRU-based RNN (128, 64 units)	Higher Recall (+9.7%), low latency	Dynamic pattern learning	GRU learns dynamic patterns, real-time, no manual features
[8, 9, 10]	Credit card, insurance, and money laundering datasets	Logistic Regression, SVM, RF, DT, MLP, LSTM, CNN, AE, GAN	RF shows strong recall/precision, deep models good for complex fraud	Most models perform better with feature selection and balancing	GAN/AE detect new patterns but need higher computational resources
[11, 12, 13, 14]	Financial big data, real-time economic and credit risk datasets	Hadoop, Spark, Storm, Impala, Deep Learning	Improved risk analysis and detection, reduced fraud	All use big data analytics for early warning and prediction	Methods differ: Storm for streaming, Spark/Hadoop for distributed analysis
[15]	Credit card transaction data	LSTM with Attention Mechanism	Best performance over traditional ML methods	Analyzes sequential data for fraud detection	Uses batch processing; our work targets real-time alerts
[16]	Synthetic and real-world financial data	PCA + Neural Network	94.39% accuracy, 94.38 F1-score in anomaly detection	Anomaly localization	Our system alerts proactively; theirs explains anomalies post hoc

5. Project Description

5.1. Proposed Solution

Financial fraud is becoming increasingly common and complex, which has motivated the development of a system that analyzes transaction data to detect unusual or suspicious activities. The system aims to improve detection accuracy, reduce false positives, and avoid unnecessary interruptions to legitimate transactions. It is designed with an emphasis on transparency, providing clear explanations for flagged cases to enhance trust among financial professionals and strengthen collaboration with technical teams. The expected outcomes include improved fraud detection, fewer unnecessary transaction blocks, greater confidence in automated tools, and real-time insights delivered through an interactive dashboard.

5.2. Methodology

This project methodology follows a standard data science project lifecycle. Data collection is the first phase, where a reliable and diverse dataset of financial transactions is compiled and used as the foundation for the project. The second phase, data understanding and preparation, includes analyzing the dataset, identifying important factors related to fraud detection, structuring and cleaning the data to ensure quality and consistency, and handling anomalies and missing values. The third step, exploratory data analysis (EDA), uses statistical analysis and visual data exploration to study transaction trends and distinguish between legitimate and fraudulent transactions. To determine the best fraud detection method, several algorithms are tested in the fourth step, model development. The model's ability to distinguish between legitimate and fraudulent transactions is used to evaluate its performance. To validate the effectiveness of the generated models in real financial systems, they are compared using accuracy and other performance indicators in the evaluation and improvement phase. Additionally, this phase also supports financial experts in assessing whether a transaction is fraudulent or legitimate.

6. Data Understanding and Exploratory Data Analysis (EDA)

Data Collection

Data collection is the first step in any data science project. It requires an extensive effort to discover reliable, useful, and accurate data that aligns with the project's objective. The data collection process is used in almost all organizations to predict future events. After this stage, the data is processed and used in the project. There are two main types of data collection: primary data and secondary data. Primary data is refers to information obtained directly from firsthand sources through interviews or experiments. It ia also known as raw data. Secondary data is information collected from various previously existing and verified sources, such as research and newspapers. The dataset used in this project is considered the secondary data due to its reliability and ease of use. It is a "Synthetic Financial Dataset for Fraud Detection," adapted from Kaggle [17]. It was designed to simulate mobile money transactions based on real-world data. The dataset includes multiple transaction types, including payments, transfers, cash withdrawals, and debit transactions. This dataset provides key details, including `step`, `type`, `amount`, `nameOrig`, `oldbalanceOrg`, `newbalanceOrig`, `nameDest`, `oldbalanceDest`, and `newbalanceDest`. It also contains (`isFraud`), indicating whether a transaction is fraudulent (0: non-fraudulent, 1: fraudulent), and (`isFlaggedFraud`), which indicates whether the system has classified the transaction as suspicious. Because it enables the use of machine learning techniques to detect suspicious transaction patterns, this structured dataset is valuable for fraud detection research.

Data Description

The dataset used in this analysis comprises approximately 6 million transaction records, with each entry representing an individual financial transaction. It includes details on the timestamp (represented by the `step` variable), transaction types, transaction amounts, customer identifiers, and balances of sender and receiver accounts before and after transactions. Additionally, two columns labeled `isFraud` and `isFlaggedFraud` serve as indicators

of fraudulent activity, providing target variables for machine learning classification tasks, meaning they are the outcomes the model aims to predict based on the provided the dataset is further illustrated through Figures 1 , 2 .

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	97225.000000	9.722500e+04	9.722500e+04	9.722500e+04	9.722500e+04	9.722400e+04	97224.000000	97224.0
mean	8.456817	1.724217e+05	8.793150e+05	8.956148e+05	8.792683e+05	1.182315e+06	0.001173	0.0
std	1.833480	3.419651e+05	2.689865e+06	2.727826e+06	2.403354e+06	2.802840e+06	0.034223	0.0
min	1.000000	3.200000e-01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000	0.0
25%	8.000000	9.893120e+03	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000	0.0
50%	9.000000	5.208135e+04	1.994700e+04	0.000000e+00	2.080800e+04	4.894480e+04	0.000000	0.0
75%	10.000000	2.103607e+05	1.863453e+05	2.107046e+05	5.853365e+05	1.051531e+06	0.000000	0.0
max	10.000000	1.000000e+07	3.379739e+07	3.400874e+07	3.400874e+07	3.397234e+07	1.000000	0.0

Figure 1: Descriptive Statistics

```
[ ] #1. Check the dataset shape (rows & columns)
    df.shape  # Returns number of rows and columns

(97225, 11)

    # 2. Get information about columns and data types
    df.info()  # Displays column types and missing values

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 97225 entries, 0 to 97224
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   step            97225 non-null  int64
 1   type            97225 non-null  object
 2   amount          97225 non-null  float64
 3   nameOrig        97225 non-null  object
 4   oldbalanceOrg   97225 non-null  float64
 5   newbalanceOrig  97225 non-null  float64
 6   nameDest        97225 non-null  object
 7   oldbalanceDest  97225 non-null  float64
 8   newbalanceDest  97224 non-null  float64
 9   isFraud         97224 non-null  float64
 10  isFlaggedFraud  97224 non-null  float64
dtypes: float64(7), int64(1), object(3)
memory usage: 8.2+ MB
```

Figure 2: Size and Structure of Dataset.

Data Dictionary

Table 2: Data Dictionary for financial fraud detection dataset.

Column Name	**Data Type**	**Description**
step	Integer	Represents the number of time steps (e.g., hourly intervals) from the start of data collection.
type	Categorical	Type of the transaction (e.g., CASH_OUT, TRANSFER, PAYMENT, DEBIT, CASH_IN).
amount	Float	Transaction amount involved in the financial process.
nameOrig	String	Identifier for the customer who initiated the transaction.
oldbalanceOrg	Float	Sender's account balance before the transaction.
newbalanceOrig	Float	Sender's account balance after the transaction.
nameDest	String	Identifier for the recipient of the transaction.
oldbalanceDest	Float	Recipient's account balance before receiving funds.
newbalanceDest	Float	Recipient's account balance after receiving funds.
isFraud	Binary (Integer)	Indicates whether the transaction is fraudulent (0: Non-fraudulent, 1: Fraudulent).
isFlaggedFraud	Binary (Integer)	Indicates whether the transaction was flagged as potentially fraudulent (0: No, 1: Yes).

Data Exploration

To understand the structure and distribution of the dataset, Exploratory Data Analysis (EDA) was conducted. Several statistical and visual techniques were applied, including box plots, bar charts, line plots, and a correlation matrix. These techniques were used to analyze the distribution of transaction types, transaction amounts, fraud trends, and correlations between different features.

Transaction Type Distribution: The distribution of transaction types reveals that PAYMENT and CASH_OUT transactions are the most frequent. However, TRANSFER transactions, although less frequent, have a higher fraud rate. As shown in Figure 3 this highlights that certain transaction types are more prone to fraudulent activities, which can be used to improve fraud detection strategies.

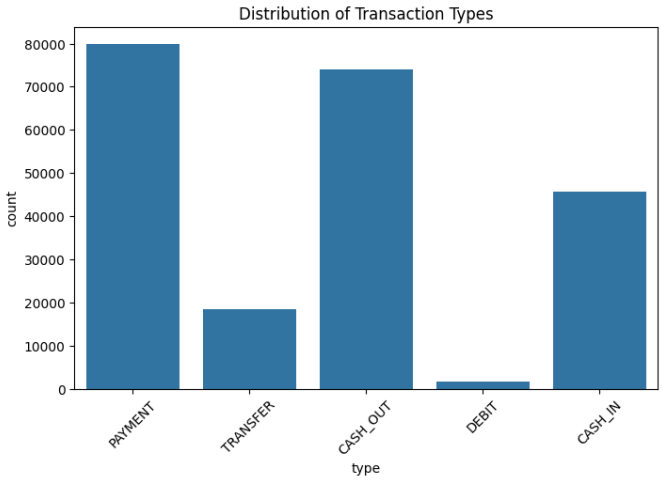

Figure 3: Distribution of Transaction Types

Transaction Amount Distribution: The distribution of transaction amounts shown in Figure 4 indicates that most transactions involve relatively small amounts. However, a small number of transactions involve exceptionally high amounts, which may represent outliers. Fraudulent transactions, represented by the orange color, tend to involve higher amounts compared to legitimate transactions, which are shown in blue. Legitimate transactions primarily cluster around lower amounts, as seen in the chart. This distinction highlights how fraudulent activity often involves larger transactions than legitimate ones.

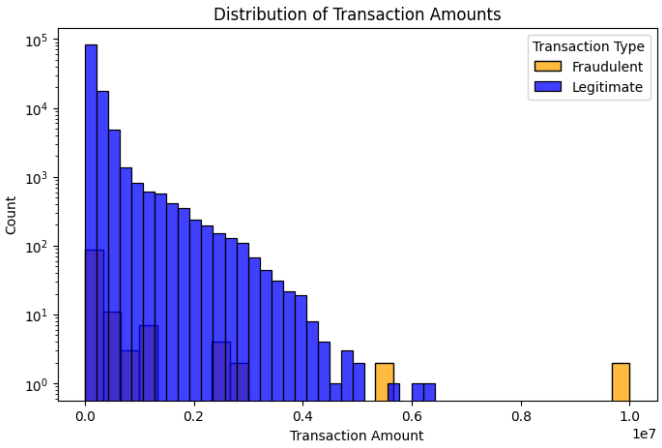

Figure 4: Distribution of Transaction Amounts

Fraud Trend Over Time: To better understand how fraudulent activity evolves, we also examine how fraud levels change over time. As shown in Figure 5, fraudulent transactions exhibit distinct patterns and notable spikes at certain hours (for example, hours 6 and 7), reflecting the necessity for time-series analysis to detect fraud early.

Figure 5: Fraud Trends Over Time

Fraud Distribution Across Transaction Types: The distribution of fraudulent transactions across different transaction types reveals interesting patterns. As shown in Figure 6, fraudulent transactions predominantly occur in CASH_OUT and TRANSFER transaction types. This distribution pattern indicates that certain transaction types may be more vulnerable to fraudulent activities, and this can help enhance fraud detection systems by focusing on these more prone transaction types.

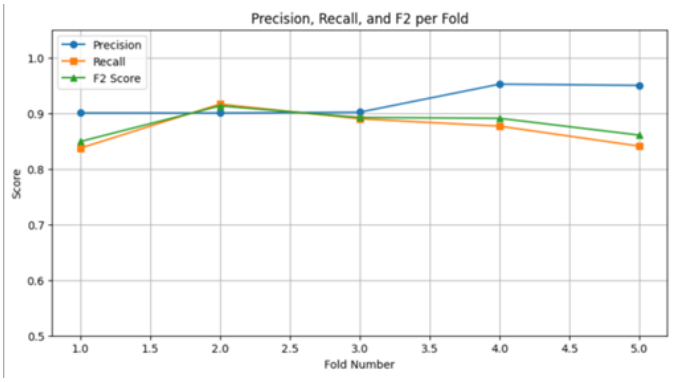

Figure 6: Fraud Distribution Across Transaction Types

Fraudulent vs. Non-Fraudulent Transactions: The dataset shows a significant imbalance between fraudulent and non-fraudulent transactions, with **99.87%** of transactions being non-fraudulent and only **0.13%** being fraudulent, as shown in Figure 7. This extreme class imbalance poses challenges for fraud detection models.

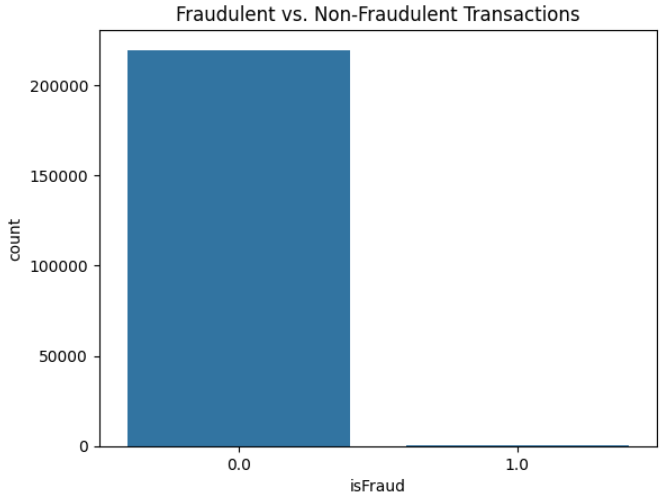

Figure 7: Fraudulent vs. Non-Fraudulent Transactions

Correlation Between The Amount And The Type Of Transaction: The correlation matrix in **Figure 8** reveals a clear relationship between the transaction type and the amount value. The strongest correlation observed was the strong positive relationship between TRANSFER operations and the amounts, with a value of 0.539**.** This indicates that transfer operations are often associated with large sums. In contrast, CASH IN and DEBIT operations showed relatively weak to negative correlations with the amount. These results suggest that transaction types linked to large amounts, such as TRANSFER, may be more vulnerable to manipulation or fraud, while small-sum operations, like DEBIT, may not be as risky.

Attributes	type = P...	type = T...	type = C...	type = D...	type = C...	amount
type = PAY...	1	-0.214	-0.531	-0.059	-0.375	-0.397
type = TRA...	-0.214	1	-0.224	-0.025	-0.158	0.539
type = CAS...	-0.531	-0.224	1	-0.062	-0.392	0.071
type = DEBIT	-0.059	-0.025	-0.062	1	-0.044	-0.048
type = CAS...	-0.375	-0.158	-0.392	-0.044	1	0.022
amount	-0.397	0.539	0.071	-0.048	0.022	1

Figure 8: Correlation Matrix Between Transaction Type and Amount

Data Quality

Upon initial inspection, we used the `df.isnull().sum()` function to check for missing values and confirmed that all attributes have complete data. We also checked for duplicate rows using the `df.duplicated().sum()` function and found no duplicate entries. Additionally, inconsistent transactions were identified by checking cases where the `oldbalanceOrg` was equal to `newbalanceOrig` and the `amount` was greater than zero. This identified 30,278 invalid transactions. The dataset exhibits a class imbalance, with fraudulent transactions representing only **0.13%**.

Insights and Future Directions

Fraudulent transactions account for only 0.13% of the total transactions, leading to a significant class imbalance. This imbalance is particularly evident in `CASH_OUT` and `TRANSFER` transactions. These types also tend to involve larger amounts. We suggest applying resampling techniques and building ML models that account for the imbalance. Further investigation into anomalies in transaction amounts is also encouraged.

7. Data Preprocessing

Removal of Unwanted Observations

As an initial step in data preparation, we focused on eliminating redundant or irrelevant data to improve dataset quality and model efficiency. The dataset was checked for duplicate rows; however, no duplicates were found in our case. Furthermore, irrelevant columns such as `nameOrig` and `nameDest`, which represent anonymized account names, were excluded from model training because in fraud detection, high data quality and proper preparation are crucial to detect rare fraudulent patterns accurately and avoid misleading the model. These columns

were preserved in the original dataset for later use in post-prediction analysis to identify the source of fraudulent activity, but were not used during model learning to prevent overfitting or data leakage.

Fixing Structural Errors

To ensure data consistency and avoid potential issues during model training, several structural adjustments were applied. First, extra spaces in column names were removed using string strip functions to maintain clean and uniform column headers. Additionally, the `type` column was standardized by converting all values to lowercase and stripping any leading or trailing spaces. This step was crucial to avoid treating variations like `CASH_OUT`, `cash_out`, or `CASH_OUT` as different categories. After these adjustments, the column names were reviewed to confirm they were correctly formatted. These changes helped improve the reliability and clarity of the dataset, ensuring it was well-prepared for downstream machine learning tasks.

Managing Unwanted Outliers in Fraud Detection

In many data analysis projects, outlier removal techniques are applied to improve data quality and reduce noise that may affect predictive model performance. However, in the context of fraud detection in financial transactions, removing outliers was deliberately avoided due to their significance in identifying fraudulent activities. Fraudulent transactions are often accompanied by unusual financial movements, such as withdrawing exceptionally large amounts compared to previous transactions for the same account. Since outliers may serve as a **key indicator of fraud**, removing them could result in losing these crucial signals, thereby reducing the model's accuracy in detecting fraudulent activities. Instead of eliminating outliers, **contextual analysis** was considered, where each transaction is evaluated based on the account's historical transactions to determine its legitimacy. A **more dynamic approach** can also be implemented, such as comparing current transaction amounts with expected thresholds derived from past withdrawal patterns, rather than relying on a fixed statistical rule like the **Interquartile Range (IQR)**. Based on these considerations, the outlier removal step was excluded to ensure that **valuable data indicative of fraudulent activities is retained**, ultimately enhancing the model's ability to achieve its primary goal.

Handling Missing Data

We used the `isnull().sum()` function to check the dataset for missing values in order to guarantee its integrity and usability.The total number of missing (null) entries in each dataset column is determined using this method. Because missing data can introduce bias and affect how well machine learning models perform, it is an essential step in the data preparation phase.Inspection revealed that there were no missing values in any of the dataset's columns. This finding suggested that the dataset was comprehensive and well-prepared. Therefore, additional imputation methods, like using predictive imputation models or filling in missing values with the mean or median, were not required.

Data Transformation, Scaling, and Normalization

We transformed, scaled, and normalized the data to improve model performance. Since the Transaction Amounts feature (**amount**) contains very large values compared to the Time Steps feature (**step**) , the model may prioritize these features. We used StandardScaler from the sklearn.preprocessing library to address the problem. This was done by standardizing the data distribution and transforming the values to a uniform range by standardizing the numeric values, giving them a mean of 0 and a standard deviation of 1. We first separated the features (X) we use to help us predict the target outcome (y, **isFraud**). We then applied the scale to features containing only numeric data to avoid applying it to categorical data (such as **type**). After normalization, we converted the data back to the pandas DataFrame format for use with SMOTE later. This transformation and scale process ensures that each feature contributes fairly to the model's training, reducing bias toward features with large values and improving model accuracy.

Why class-weighted XGBoost outperforms SMOTE-per-fold

In Senior Project 1 (SP1), SMOTE was initially applied inside each training fold to address the extreme class imbalance in the dataset while preventing data leakage. This approach provided a useful baseline for understanding the behavior of oversampling techniques in fraud detection.

However, in Senior Project 2 (SP2), and after conducting deeper experimentation and evaluation, we transitioned to using **class-weighted XGBoost** as the primary

method for handling imbalance. Even when SMOTE is applied correctly inside each fold, class-weighted XGBoost still demonstrates superior performance. This preference is supported by several factors:

1. **Loss-level rebalancing:** Class weighting rebalances the problem directly at the loss function level, enabling the model to optimize margins on real data rather than relying on synthetic minority samples.
2. **Non-linear fraud pattern modeling:** Boosted trees naturally capture complex interactions and rare fraud behaviors without depending on synthetic neighbors generated by SMOTE.
3. **Reduced distribution shift:** Training on real-only data minimizes train–test distribution discrepancies and results in more stable calibration and threshold tuning.
4. **Lower pipeline complexity:** The approach avoids SMOTE's sensitivity to hyperparameters such as the number of neighbors, random seeds, and minority topology, making it more suitable for production deployment.
5. **Superior experimental performance:** In our experiments, class-weighted XGBoost achieved higher and more stable PR-AUC and F2-scores across folds compared to SMOTE-per-fold, confirming a better precision–recall trade-off for operational fraud detection.

This progression from SMOTE in SP1 to class-weighted XGBoost in SP2 reflects the natural evolution of our system toward a more robust, stable, and deployment-ready fraud detection model.

8) Experiments Setup and Tools

The experimental setup was implemented in a Python-based environment using Google Colab, which provides scalable GPU and CPU resources suitable for data-intensive machine learning tasks. The development relied on several scientific libraries to support data analysis, model training, evaluation, and interpretability, including Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, XGBoost, and SHAP, converting categorical attributes into numerical form using One-Hot Encoding, and applying StandardScaler and RobustScaler to normalize numerical attributes.

Libraries and Tools Explanation

To clarify the role of each tool used in the experiments, the following summarizes the purpose of all libraries included in the development process:

- **Pandas:** Used for loading, cleaning, and transforming large tabular datasets.
- **NumPy:** Provided efficient numerical operations and array computations essential for model training.
- **Matplotlib and Seaborn:** Utilized to generate EDA visualizations such as distributions, trends, and performance plots.
- **Scikit-learn:** Supplied preprocessing tools (OneHotEncoder, StandardScaler, RobustScaler), model evaluation methods, and baseline algorithms including Logistic Regression and Random Forest.
- **XGBoost:** Implemented as the main model due to its superior handling of class imbalance through `scale_pos_weight` and strong performance on non-linear patterns.
- **SHAP:** Enabled interpretation of the XGBoost model's predictions by identifying the most influential features contributing to fraud detection.
- **OneHotEncoder:** Converted categorical data into numerical vectors suitable for machine learning algorithms.
- **StandardScaler:** Normalized features with normally distributed ranges.
- **RobustScaler:** Scaled features affected by outliers to maintain stability during training.

All experiments were conducted in a controlled environment using Python 3.10 on Google Colab, leveraging open-source libraries to ensure reproducibility and transparency of results.

9) Model architectures, initial parameters and selection criteria

We began with a simple question: which model can detect rare, costly fraud without overwhelming analysts with false positives? To answer it, we compared three tabular classifiers—Logistic Regression, Random Forest, and XGBoost—under the same stratified 5-fold protocol. A consistent pattern emerged: Logistic Regression underfit

the non-linear structure and missed many frauds; Random Forest achieved high precision but left recall on the table; XGBoost delivered the best balance, handling severe class imbalance while capturing salient feature interactions.

Stratified Cross-Validation and Threshold Selection

The Stratified 5-fold Cross-Validation protocol was essential to ensure the model was trained and evaluated on uniformly distributed data splits across all folds. This procedure was crucial for two main reasons:

1. **Ensuring Fair Distribution:** Given the severe class imbalance, stratification guarantees that the ratio of fraud to non-fraud cases remains **nearly identical** in each training and testing fold, preventing bias. The table below illustrates the robust distribution of cases in the training data for each fold:

Fold Index	Fraud_train	NonFraud_train
0	907	835556
1	907	835556
2	907	835556
3	907	835556
4	908	835556

Table 3: Distribution of Fraud and Non-Fraud samples across training folds.

2. **Stability Testing:** Cross-validation allows us to test the model's performance on varied distributed data, confirming that the model's performance is **stable** and not reliant on a specific data subset.

To determine the **Optimal Operating Threshold** and validate the suitability of XGBoost, we optimized for the **$F2$ Score** within the cross-validation folds. The $F2$ metric places a higher weight on **Recall** over Precision, which is paramount in fraud detection to mitigate the substantial costs associated with missed fraud cases.

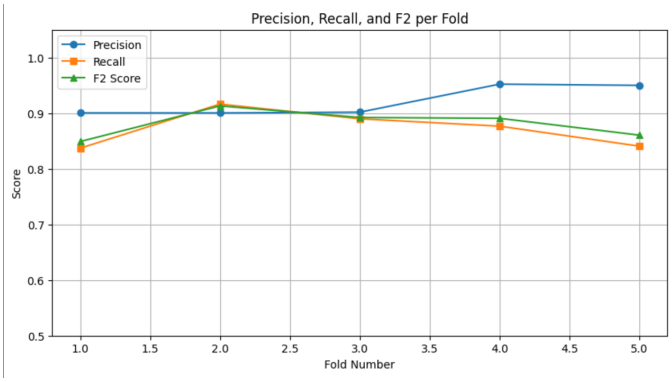

Figure 9: Precision, Recall, and F2 Score for each fold.

Model Selection Criteria

Model selection was primarily driven by the mean $PR-AUC$(Precision-Recall Area Under the Curve.) across folds, a metric highly appropriate for class imbalance scenarios. Furthermore, due to the high cost of missing fraud, we also optimized for Recall using the mean $F2$ Score at the operating threshold. Beyond quantitative metrics, we required the model to exhibit:

1. **Fold-to-fold Stability:** Demonstrated by consistent confusion-matrix patterns across the folds (as shown in Figure 9).
2. **Interpretability:** Via SHAP (SHapley Additive exPlanations) values, ensuring the model's logic aligns with domain expectations.

Initial Parameters

For XGBoost we adopted a robust baseline with light tuning: 300 trees, learning rate 0.10, maximum depth 6, subsample 0.8, colsample_bytree 0.8, histogram-based tree growth, and `aucpr` as the training metric, all with a fixed seed (42). Rather than oversampling, we used the model's internal class weighting: in each fold, `scale_pos_weight = #NonFraud / #Fraud` so the minority class receives proper emphasis.

Selection Criteria

Model selection was driven primarily by mean PR-AUC across folds (appropriate under class imbalance). Because missing fraud is costly, we also optimized for recall using mean F2 at the operating threshold. Beyond metrics, we required fold-to-fold stability (consistent confusion-matrix patterns) and interpretability via SHAP.

Cross-validation Performance

Figure 10 summarizes fold-wise performance. ROC–AUC remains near 1.00 for all folds, while PR–AUC is stable around 0.96–0.97 (mean PR–AUC ≈ 0.9668). This plot evidences both high separability and stability of the class-weighted XGBoost across the 5 folds, supporting its selection as the primary model. Complementary fold-level operating points (chosen by maximizing F2) yielded mean F2 ≈ 0.9147, confirming a recall-oriented operating regime with controlled false positives.

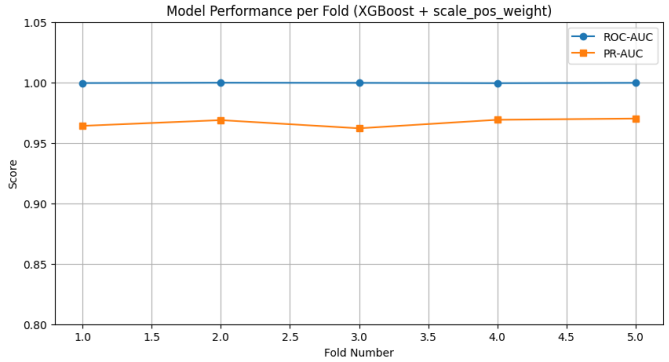

Figure 10: Performance across 5 stratified folds for class-weighted XGBoost.

Time-feature engineering and ablation. The raw `step` (a monotonically increasing time index) does not encode daily periodicity (e.g., 23:00 and 00:00 are adjacent in time but numerically far). We replaced it with features the model can use: relative time (`step_rel`), day and hour buckets, sinusoidal encodings (`step_sin`, `step_cos`) to capture circularity, and a peak-hour flag. An ablation showed that removing the raw `step` while keeping these derived features preserves performance (PR-AUC 0.982 vs. 0.981; Recall 0.974 vs. 0.973) and yields clearer explanations, so we retained the derivatives and dropped the raw index.

Baselines and variants

As a compact numerical summary, the test-set results are reported in Table 4. **Logistic Regression (class-weighted + threshold tuning).** We trained a linear classifier with `class_weight=balanced` and tuned the decision threshold on validation folds. Despite being simple and fast, the linear boundary could not capture non-linear structure; fraud recall remained low (**F1**=0.57, **recall**=0.47, **precision**=0.71).

Random Forest (plain). A non-linear ensemble of trees (hundreds of estimators, default class weights) yielded very high precision but lower recall (**F1**=0.88, **recall**=0.80, **precision**=0.99), which is typical for RF under severe imbalance unless the operating point or class weights are adjusted.

Balanced Random Forest (undersampling). Rebalancing each bootstrap sample pushed recall extremely high but collapsed precision (**F1**=0.14, **recall**=0.97, **Balanced Random Forest (undersampling).** Rebalancing each bootstrap sample pushed recall extremely high but collapsed precision (**F1**=0.14, **recall**=0.97, **precision**=0.07), producing too many false alarms to be actionable.

XGBoost with internal class weighting. Gradient-boosted trees with `scale_pos_weight` set per fold to #NonFraud/#Fraud struck the best overall balance (**PR–AUC**=0.9569, **F1**=0.89, **recall**=0.85, **precision**=0.94; **ROC–AUC** ≈ 0.9997). This reshapes the loss to counter class imbalance without altering the data distribution.

Soft-voting ensemble (XGB + RF). Combining calibrated probabilities from XGBoost and Random Forest in a weighted soft vote was stable but did not surpass standalone XGBoost (**PR–AUC** ≈ 0.9408, **F1** ≈ 0.87, **recall** ≈ 0.84, **precision** ≈ 0.90). Pushing the operating point toward F_2 increased recall to ≈ 0.90 with the expected precision drop to ≈ 0.76 (**F1** ≈ 0.83). We therefore retained a single, well-calibrated XGBoost as the primary model.

Explainability (Figure 11). SHAP served not only as a diagnostic but also as a selection criterion. The global SHAP summary (Figure 11) highlights drivers consistent with domain expectations—`oldbalanceOrg`, `newbalanceOrig`, and `amount`, followed by transaction type and time-derived features—indicating that the model's logic mirrors real transactional behavior. At the local level, SHAP turns each alert into a concise reason code (e.g., amount↑, oldbalanceOrg↓, newbalanceOrig↑), which shortens investigations and supports audits.

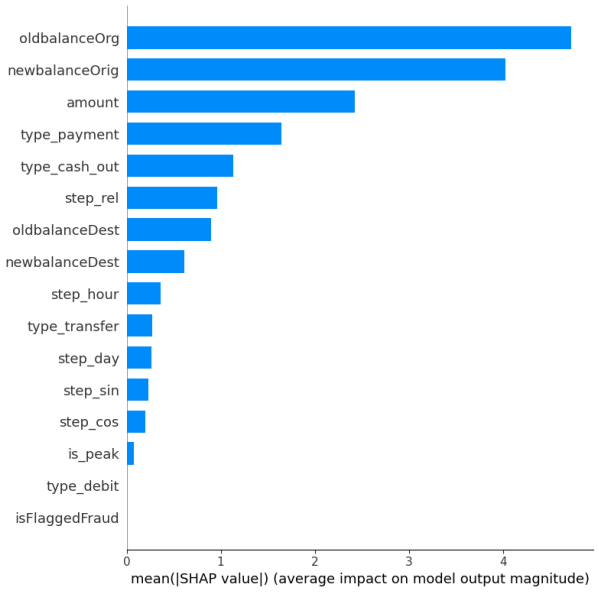

Figure 11: SHAP feature importance: *oldbalanceOrg*, *newbalanceOrig*, and *amount* are the most influential; type and time features follow, matching domain expectations.

Under these quantitative and qualitative checks, we adopted a single, well-calibrated XGBoost with a recall-oriented threshold as the model of record.

Table 4: Compact comparison of candidate models on the test set.

Model	**Technique**	**PR-AUC**	**F1**	**Recall**	**Precision**	**Thresh.**
Logistic Regression	class_weight=balanced + threshold tuning	–	0.57	0.47	0.71	tuned
Random Forest (plain)	default (no rebalance)	–	0.88	0.80	0.99	default
Balanced Random Forest	undersampling per bootstrap	–	0.14	0.97	0.07	default
XGBoost	scale_pos_weight (per fold)	**0.9569**	**0.89**	**0.85**	**0.94**	F1@
Ensemble (XGB+RF)	weighted soft vote	0.9408	0.87	0.84	0.90	F1@
Ensemble (XGB+RF)	weighted soft vote	0.9408	0.83	0.90	0.76	F2@

10) Model Deployment

The final stage of the data science lifecycle involves model deployment, where the developed fraud detection model is transitioned from analytical experimentation into a functional, real-world application. In this phase, the proposed XGBoost model was successfully integrated into an advanced Power BI dashboard that enables dynamic interaction, real-time monitoring, and comprehensive performance visualization.

This upgraded dashboard goes beyond standard metrics, offering an in-depth analytical view of the model's predictive capabilities and transaction-level insights. It bridges the gap between technical analysis and financial decision-making, empowering analysts to interpret results and take proactive actions based on visual evidence.

Dashboard Visualization

Figure 12: Fraud Detection Dashboard presenting key performance metrics, SHAP-based feature importance, fraud loss prevention results, risk-level distribution, and time-based trends of fraud versus legitimate transactions.

Dashboard Visualization Description

Overall Performance Indicators

The top section of the dashboard in Figure 12 presents key performance indicators such as the F1-score, recall, and the aggregated fraud probability. These metrics summarize the model's overall predictive effectiveness and its sensitivity to fraudulent behavior.

Why Fraud Happens — SHAP Model Explanation

The SHAP analysis panel in Figure 12 illustrates the most influential features contributing to the model's fraud predictions. The bars represent the magnitude of each feature's contribution to increasing the probability of a transaction being classified as fraudulent.

The results indicate that *Unusual Source Balance* is the most significant driver of fraud, as abnormal or unexpected account balances strongly signal suspicious activity. The second most important factor is *Balance Drop After Transfer*, followed by high-value transfer operations, which are commonly associated with abnormal or risky transactions. While *Payment* and *Cash-Out* operations also contribute to fraud prediction, their impact is comparatively lower.

Overall, this panel enhances the interpretability of the model by clearly highlighting the behavioral patterns most associated with fraudulent activity.

Fraud Loss Prevention Rate

The Fraud Loss Prevention Rate in Figure 12 shows the proportion of potential financial loss that the detection system successfully prevented. The first bar represents the total estimated fraud loss, while the second bar illustrates the amount of loss avoided due to early detection. The remaining portion represents missed loss, corresponding to undetected fraudulent transactions.

The visualization demonstrates that the model prevented a significant share of the expected financial damage, indicating high effectiveness in mitigating fraud-related losses.

Fraud Risk Level Distribution

The risk distribution panel presents the classification of transactions into three risk levels: **Low**, **Medium**, and **High**. The **Low-risk** category dominates the distribution (99.68%), while only a small fraction of transactions fall into the **Medium** (0.11%) and **High** (0.21%) risk categories.

This imbalance, shown in Figure 12, reflects the natural rarity of fraudulent activities in real-world financial systems and emphasizes the importance of directing analytical resources toward high-risk cases.

Recommended Actions After Fraud Detection

The recommended actions in 12section outlines practical operational responses following the identification of suspicious or high-risk transactions:

- **Automatically block suspicious transactions.** Prevents potentially fraudulent operations from being completed while allowing safe manual review.
- **Strengthen two-factor authentication for high-risk users.** Adds additional verification layers to reduce unauthorized access.
- **Retrain the detection model regularly.** Ensures the model remains accurate and adapts to new fraud patterns.
- **Apply daily transaction limits.** Helps minimize potential financial loss in case of fraud.

Key Performance Metrics

The metrics in 12 panel displays several critical values , for illustration Cards starts from left to right and numberd :

Number of Total Transactions (1)

This card shows the total number of transactions in the dataset, including both legitimate and fraudulent entries, providing an overall view of the data volume processed by the system.

Number of Legitimate Transactions (2)

This card presents the total count of legitimate transactions, helping to highlight the proportion of safe and normal activities within the dataset.

Number of Fraudulent Transactions(3)

This card displays the number of transactions identified as fraudulent, indicating the scale of suspicious activity detected by the model.

Total Fraud Risk Score (4)

This card represents the total aggregated fraud risk score across all transactions, summarizing the overall level of risk within the dataset.

These indicators provide a concise overview of the dataset's composition and the model's overall risk assessment.

Fraud vs. Legitimate Transactions Over Time
As shown in 12the steps represent the order of transaction occurrences, enabling the observation of trends and fluctuations in fraud patterns over time.

This view helps identify unusual spikes or irregular trends that may indicate coordinated fraudulent behavior.

Discussion of Deployment Impact
The enhanced dashboard presented in Figure 12 transforms the project from a purely analytical prototype into a semi-operational fraud monitoring system. It enables continuous performance evaluation, model interpretability through SHAP-based explanations, and user-driven risk assessment.

Overall, this deployment phase demonstrates the effectiveness of integrating machine learning, explainable AI, and advanced visualization techniques to develop a transparent, data-driven, and practical early warning system for financial fraud detection.

11) Results & Discussion

1. **Performance Evaluation Metrics**
   This section discusses the performance metrics used to evaluate the effectiveness of the proposed financial fraud detection model, focusing on its ability to detect fraudulent transactions. The classification performance metrics used include Precision, recall, F1 score, F2 score, area under the curve (ROC-AUC), and area under the curve (PR-AUC). These metrics are essential for evaluating the model in cases where fraudulent transactions constitute only 0.13% of the entire dataset. The project focused on metrics that prioritize the minority category (fraud category) due to the significant imbalance in the categories. Traditional accuracy was deemed insufficient, as it can lead to inaccurate results in unbalanced cases and may even produce misleading findings.
   1. Precision, which is essential for reducing false positives, is the percentage of transactions detected as fraudulent that are indeed fraudulent.
   2. The Recall assesses the model's ability to detect all actual fraud cases, which is crucial for minimizing undetected fraud (false negatives).
   3. F1-Score: Balance between Precision and Recall.
   4. F2-Score: Increases the weight of the recall in order to identify more cases of fraud.
   5. ROC-AUC: Evaluates how well the model distinguishes between classes.

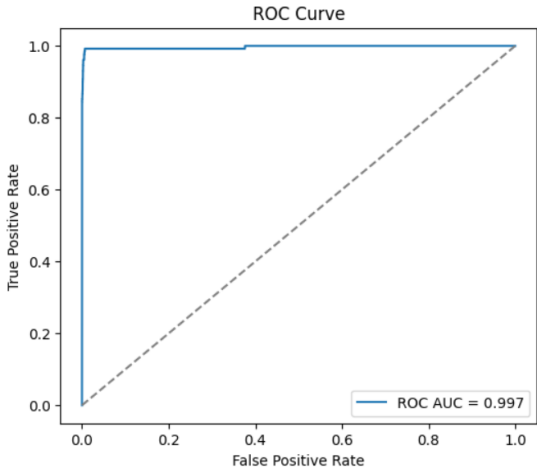

Figure 13: ROC Curve

   6. PR-AUC: Accurate, concession-focused, and reliable retrieval of unbalanced data.

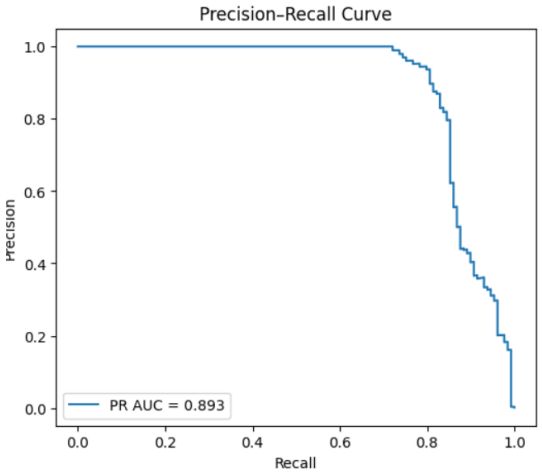

Figure 14: Precision–Recall Curve

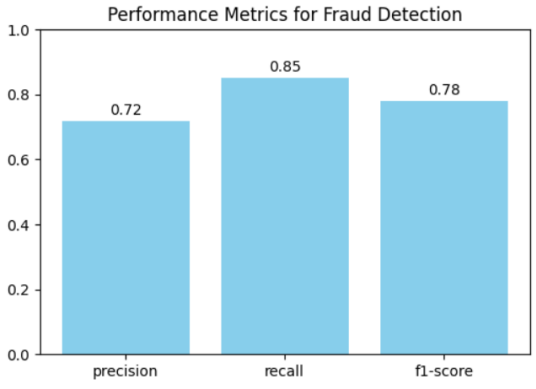

Figure 15: Performance Metrics for Fraud Detection

2. Experimental Results

The experimental results demonstrate the robustness and reliability of the proposed model in detecting financial fraud. Multiple machine learning algorithms were evaluated, including Logistic Regression, Random Forest, and XGBoost, under a stratified five-fold cross-validation setup to ensure stable performance across different subsets of the data. Each fold maintained the same ratio of fraudulent to non-fraudulent transactions, ensuring that the model's performance was not biased by data imbalance (see Figure 9 for fold distribution and setup).

The experimental findings revealed clear distinctions in performance among the models. Logistic Regression exhibited limited ability to capture complex relationships, resulting in a lower recall and an overall F1-score of 0.57. The Random Forest model achieved higher precision (0.99) and an F1-score of 0.88, but its recall value of 0.80 indicated that some fraudulent cases were still undetected. In contrast, the XGBoost model—using internal class weighting—demonstrated the most balanced and consistent results, achieving a PR-AUC of 0.9569, F1-score of 0.89, precision of 0.94, and recall of 0.85. These values confirm the model's ability to maintain high sensitivity to fraud while avoiding excessive false positives (refer to Figure 10 for comparative model performance).

Across all cross-validation folds, XGBoost maintained remarkable consistency, with an average ROC-AUC close to 1.00, PR-AUC around 0.97, and mean F2-score of approximately 0.91. These outcomes indicate strong generalization capability and a recall-oriented performance—an essential characteristic in fraud detection systems (supported by Figure 9 and Figure 10 showing fold-wise results).

Further interpretability was achieved through SHAP (SHapley Additive Explanations) analysis, which revealed that features such as *oldbalanceOrg*, *newbalanceOrig*, *amount*, and *transaction type* played the most significant roles in predicting fraud. These findings align with domain expectations, as irregularities in balance and unusually large transactions often correspond to fraudulent activity (as shown in Figure 11).

Finally, the experimental evaluation was reinforced by visualization techniques, including ROC and Precision–Recall curves, which confirmed near-perfect separability between fraudulent and legitimate classes. The integration of the trained XGBoost model into a Power BI dashboard transformed these experimental results into a practical monitoring tool that enables financial analysts to visualize risk levels, fraud probabilities, and performance metrics in real time. This demonstrates the success of the proposed early warning system in combining accuracy, transparency, and usability for real-world fraud detection (see Figures 12–14 for dashboard visualization and model performance curves).

3. Discussion

This technique, as demonstrated in the study, using **XGBoost** with internal class weighting provides a strong balance between precision and recall for financial fraud detection. Unlike simpler algorithms, it effectively handled the extreme data imbalance without relying on oversampling techniques, which reduced false positives and maintained consistent performance across all validation folds. which reduced false positives and maintained consistent performance across all validation folds.

The explainability analysis using **SHAP** confirmed that key financial attributes—such as account balances before and after transactions and transaction amount—play a major role in identifying suspicious behavior. This transparency improves the system's trustworthiness, allowing financial analysts to understand the reasoning behind each prediction.

Overall, the results indicate that integrating explainable AI with optimized machine learning can significantly enhance early fraud detection. However, future improvements could include incorporating real-time data streams and adaptive learning to ensure continuous model accuracy in changing financial environments.

4. Conclusion and Future Work

Conclusion

This project successfully developed an early warning system for financial fraud detection by leveraging machine learning, and explainable AI techniques. The system utilized a synthetic financial transactions dataset to simulate real-world payment behaviors. A class-weighted XGBoost model was implemented to handle the severe class imbalance inherent in fraud data, demonstrating superior performance compared to baseline models like Logistic Regression and Random Forest.

The final model achieved a robust and recall-oriented performance, with a mean F2-score of approximately 0.9147 and a mean PR-AUC of 0.9668 across stratified cross-validation folds. This confirms the model's high capability to detect fraudulent transactions while effectively minimizing false positives, a critical requirement in financial operations. Furthermore, the integration of SHAP (SHapley Additive exPlanations) provided global and local interpretability, revealing that features such as original balance, new balance, and transaction amount were the primary drivers of predictions, which aligns with domain expertise. This transparency fosters trust and facilitates collaboration between technical and financial teams.

The operationalization of the model through an interactive Power BI dashboard transformed the analytical prototype into a practical monitoring tool, enabling real-time risk analysis and visualization of model performance. In summary, this work underscores the effectiveness of combining advanced machine learning, explainable AI to build a transparent, reliable, and actionable early warning system for financial fraud detection.

***F**uture Work*

Future work and Limitation: The current model faces a few limitations related to data availability, real-time processing, and model adaptability. Each future step is meant to improve the current system's limitations. The following points directions for future development :

1. **using more data source**: using more data source such as geolocation information, user digital behavior, and device history, could provide a more comprehensive contextual analysis for fraud detection.
2. **Moving toward real-time detection**: in the future adopting stream processing frameworks like Apache Kafka 16 or Apache Flink would reduce detection latency, enabling true real-time fraud prevention as transactions occur.
3. **Trying More Advanced Models**: Exploring state-of-the-art sequence models, such as Transformers or hybrid architectures (e.g., CNN-LSTM), could potentially capture more complex temporal patterns and improve predictive accuracy.
4. **Handling Imbalance data better**: Investigating newer techniques for handling class imbalance, such as Generative Adversarial Networks (GANs) for synthetic data generation or Semi-Supervised Learning approaches to leverage unlabeled data, could further boost performance.
5. **Online Learning Mechanism**: Developing an online learning pipeline would allow the model to adapt continuously to evolving fraud patterns without the need for periodic full retraining, increasing the system's longevity and effectiveness.
6. **Dashboard Enhancement**: The Power BI dashboard can be extended with features like automated alert notifications, periodic performance reports, and dynamic model updating based on analyst feedback.

References

[1] S. B. Shetty, P. Kumar, A machine learning based credit card fraud detection using the ga algorithm for feature selection, Journal of Big Data 9 (2022) 56. doi:10.1186/s40537-022-00573-8.
URL https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00573-8

[2] L. Harris, Fraud detection in the financial sector using advanced data analysis techniques, ResearchGate (2024).
URL https://www.researchgate.net/publication/386111741_Fraud_Detection_in_the_Financial_Sector_Using_Advanced_Data_Analysis_Techniques

[3] A. Iqbal, R. Amin, Time series forecasting and anomaly detection using deep learning, Computers Chemical Engineering 182 (2024).
URL https://www.sciencedirect.com/science/article/abs/pii/S0098135423004301

[4] S. Ghimire, Timetrail: Unveiling financial fraud patterns through temporal correlation analysis, arXiv preprint (2023). arXiv:2308.14215.

[5] Z. Xiao, J. Jiao, Explainable fraud detection for few labeled time series data, Security and Communication Networks 2021 (2021) 1–9. doi:10.1155/2021/9941464.
URL https://doi.org/10.1155/2021/9941464

[6] A. S. G. M. S. C. A. J. T. A. Bernardo Branco, Pedro Abreu, P. Bizarro, Interleaved sequence rnns for fraud detection, Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (2020).
URL https://doi.org/10.1145/3394486.3403361

[7] G. Moschini, R. Houssou, J. Bovay, S. Robert-Nicoud, Anomaly and fraud detection in credit card transactions using the arima model, Engineering Proceedings (2021).
URL https://doi.org/10.3390/engproc2021005056

[8] W. Hilal, Anomaly detection in financial fraud: Machine learning approaches and challenges, Expert Systems with Applications 193 (2022) 116429. doi:10.1016/j.eswa.2021.116429.
URL https://doi.org/10.1016/j.eswa.2021.116429

[9] S. A. Gadsden, Anomaly detection in financial fraud: Machine learning approaches and challenges, Expert Systems with Applications 193 (2022) 116429. doi:10.1016/j.eswa.2021.116429.
URL https://doi.org/10.1016/j.eswa.2021.116429

[10] J. Yawney, Anomaly detection in financial fraud: Machine learning approaches and challenges, Expert Systems with Applications 193 (2022) 116429. doi:10.1016/j.eswa.2021.116429.
URL https://doi.org/10.1016/j.eswa.2021.116429

[11] Y. Zhang, Y. Xu, Z. Y. Dong, Z. Xu, K. P. Wong, Intelligent early warning of power system dynamic insecurity risk: Toward optimal accuracy-earliness tradeoff, IEEE Transactions on Industrial Informatics 13 (5) (2017) 2544–2554.

[12] E. M. Akhmetshin, V. L. Vasilev, Control as an instrument of management and institution of economic security, Academy of Strategic Management Journal 15 (2016) 1.

[13] S. Piramuthu, Feature selection for financial credit-risk evaluation decisions, INFORMS Journal on Computing 11 (3) (1999) 258–266.

[14] X. L. Z. Lv, K.-K.-R. Choo, E-government multimedia big data platform for disaster management, Multimedia Tools and Applications 77 (8) (2018) 10077–10089.

[15] B. E. O. J. J. I. Benchaji, S. Douzi, Enhanced credit card fraud detection based on attention mechanism and lstm deep model, Journal of Big Data 8 (2021) 151. doi:10.1186/s40537-021-00541-8.

[16] N. M. M. T. S. Crépey, N. Lehdili, Anomaly detection in financial time series by principal component analysis and neural networks, Algorithms 15 (10) (2022) 385. doi:10.3390/a15100385.
URL https://doi.org/10.3390/a15100385

[17] S. Eedala, Financial fraud detection dataset, https://www.kaggle.com/datasets/sriharshaeedala/financial-fraud-detection-dataset, accessed: December 3, 2025 (2023).